HW #2: JS and NodeJS

Contents

Part 1: JavaScript basics (review) 1
1.1 Intro to JS 1
1.2. Standard JS functions 2
1.3. Function Expressions 2
1.4. Arrow Functions 3
1.5. Constructor Functions 4
1.6. Default Parameters 5
1.7. Destructuring 5
Part 2: Intro to Asynchronous JS 6
2.1 Fundamentals of Asynchronicity 6
2.2 Event loop 7
2.3 Promises 8
2.4 Promise Chaining 9
2.5 Promise API 10
2.6. Async/Await 11
2.7. Error Handling 11
2.8. Parallel Execution with Promise.all(): 12
2.9. Fetch API 13
2.10. Advanced features 13
Part 3: Intro to Node.js 14
3.1 Introduction to Node.js and NPM 14
3.2. Express Framework 16

In this lab, you will explore JavaScript programming language and its latest features, will run it both on a browser and in the NodeJS server-side environment.

Part 1: JavaScript basics (review)

1.1 Intro to JS

JavaScript (JS) is a versatile programming language primarily used to add **interactivity** and **dynamic behavior** to websites. It allows developers to create features like animations, pop-up menus, form validation, and even small web apps. JavaScript can **run in web browsers**, interpreting code to modify the content and appearance of web pages on the fly. It's a core technology of the web alongside HTML (structure) and CSS (styling), enabling developers to build modern, engaging online experiences. Thanks to environments like **Node.js**, JavaScript can now **run on servers**, enabling the development of backend

applications, command-line tools, and even desktop applications. The language gets updated every year: https://tc39.es/ecma262/multipage/#sec-intro.

1.2. Standard JS functions

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions, https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-function-definitions

Functions are reusable blocks of code that perform a specific task. They help organize your code, make it more readable, and prevent repetition. There are several ways to declare a function **in JS**:

Create a folder on your computer called yk_js (your initial), select and open it in VSC. Create a file functions.js and try the following snippets and develop those under **Practice**.

Paste this code to your .js file and execute

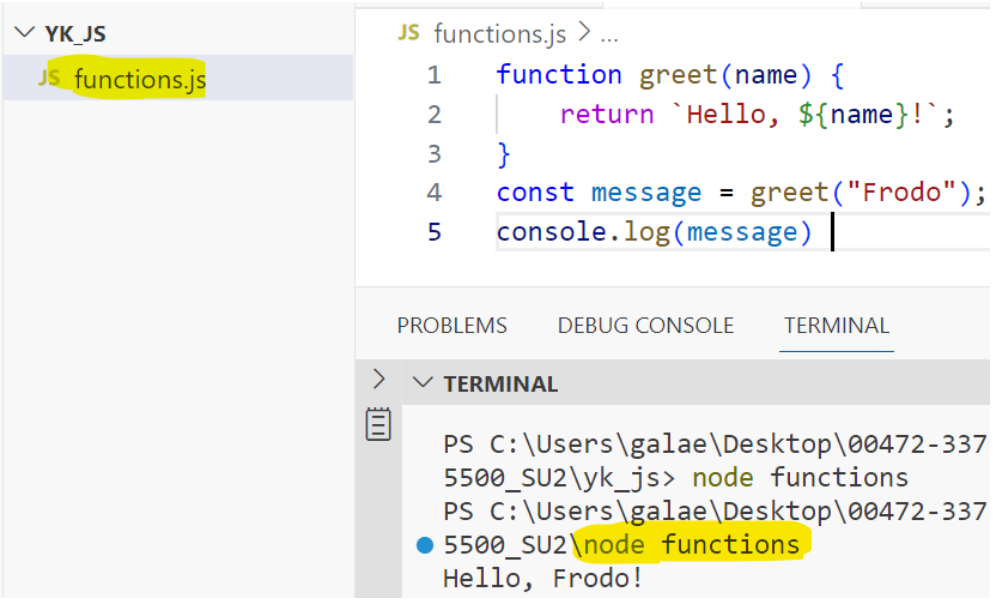
Function Declaration:

```
function greet(name) {
  return `Hello, ${name}!`;
}
```

Calling a Function:

```
const message = greet("Frodo"); // message will be "Hello, Frodo!"
```

right-click. Open in integrated terminal, run



Practice: Write a **function** declaration named **countTrolls** that takes a **number** parameter and returns a string stating how many trolls there are in Mordor. Print your result to console.

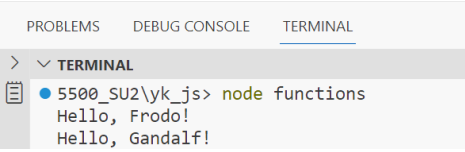
Practice: Write a **function** declaration named **reverseName** that takes a string parameter (a character's name) and returns the reversed name. Print your result to console. Use LLMs to assist you and explain the code (if needed)

1.3. Function Expressions

A **function expression** is created by **assigning a function to a variable**. This can be done using either the **function** keyword or arrow functions.

```
const greet2 = function(name) {
  return `Hello, ${name}!`;
};
```

```
8  const greet2 = function(name) {
9      return `Hello, ${name}!`;
10   };
11   const message2 = greet2("Gandalf");
12   console.log(message2)
```



Practice: Write a **function** expression named **defeatOrcs** that takes two parameters (number of orcs and number of elves) and returns a string stating whether the elves can defeat the orcs. Print your result to console.

Hint: it depends on how many orcs and elves are out there, assume hundreds of them

Write a **function** expression named **isRingBearer** that takes a character's name and returns true if the name is "Frodo" or "Sam", and false otherwise. Print your result to console.

1.4. Arrow Functions

Arrow functions provide a **concise** and **more readable** syntax for writing function expressions. Introduced in ES6 (ECMAScript 2015), they have some distinct characteristics and advantages, especially regarding the **this** keyword.

Arrow functions are defined using the **=>** syntax. Here are a few examples:

Single parameter, single expression:

```
const bye = name => `Bye-bye, ${name}!`;
console.log(bye ("Frodo")); // Bye-bye, Frodo!
```

Multiple parameters:

```
const add = (a, b) => a + b;
console.log(add(1, 2)); // 3
```

No parameters:

```
const getRingBearer = () => "Frodo Baggins";
console.log(getRingBearer()); // Frodo Baggins
```

Multiple statements:

When the function body has multiple statements, you must **use curly braces {}** and explicitly use the **return** statement:

```
const battle = (warrior, weapon) => {
  console.log(`${warrior} wields a ${weapon}.`);
  return `${warrior} fights bravely!`;
};
console.log(battle("Aragorn", "sword")); // Aragorn wields a sword. Aragorn fights bravely!
```

```
14  const bye = name => `Bye-bye, ${name}!`;
15  console.log(bye ("Frodo")); // Bye-bye, Frodo!
16  const add = (a, b) => a + b;//Multiple parameters:
17  console.log(add(1, 2)); // 3
18  const getRingBearer = () => "Frodo Baggins";//No pa
19  console.log(getRingBearer()); // Frodo Baggins //Mu
20  //When the function body has multiple statements, y
21  const battle = (warrior, weapon) => {
22    console.log(`${warrior} wields a ${weapon}.`);
23    return `${warrior} fights bravely!`;
24  };
```

PROBLEMS DEBUG CONSOLE TERMINAL

TERMINAL

```
Bye-bye, Frodo!
3
Frodo Baggins
Aragorn wields a sword.
Aragorn fights bravely!
```

1.5. Constructor Functions

*Such functions are used to **create objects**. When called with **new**, they **act like a template for new objects**.*

```
function Elf(name, weapon) {
  this.name = name;
  this.weapon = weapon;

  this.describe = function() {
    console.log(`${this.name} wields a ${this.weapon}.`);
  };
}
```

Create Instances and Call Methods:

```
const legolas = new Elf("Legolas", "bow and arrow");
const tauriel = new Elf("Tauriel", "dagger");

legolas.describe(); // Logs "Legolas wields a bow and arrow."
tauriel.describe(); // Logs "Tauriel wields a dagger."
```

Arrow functions cannot be used with the **new** keyword.

Practice: Convert Traditional Function to Arrow Function and test it. Print your result to console.

```
function travelTo(location) { //outputs Traveling to Mordor
  return `Traveling to ${location}`;
}
```

Practice: Write an **arrow function** to **double** each element in an array of numbers: const numbers = [1, 2, 3, 4];

Practice: Write an arrow function to filter out even numbers from an array of numbers: const numbers2 = [1, 2, 3, 4, 5, 6];

Practice: Write an arrow function that takes an array of names and prints each name with a message indicating they are part of the Fellowship: hint: members are "Frodo", "Sam", "Merry" and "Pippin"

1.6. Default Parameters

Default parameters in JavaScript allow you to initialize function parameters with **default values** if no value or **undefined** is provided. This feature simplifies function definitions and improves code readability and maintainability by eliminating the need for additional logic to handle missing arguments.

Syntax

```
function functionName(parameter1 = defaultValue1, parameter2 = defaultValue2, ...) {
  // Function body
}
```

When calling the function, if an argument is not provided or is undefined, **the default value will be used**.

```
function greet(name = "Stranger") {
  return `Hello, ${name}!`;
}
console.log(greet()); // "Hello, Stranger!"
console.log(greet ("Frodo")); // "Hello, Frodo!"
```

Practice: Write a function named **describeBattle** that takes two parameters, location and enemies (**default** value "orcs") and returns a description of the battle.

1.7. Destructuring

Destructuring is a convenient way of **extracting values from arrays or properties from objects** into distinct variables. This feature allows you to write more concise and readable code, especially when working with complex data structures. Array destructuring allows you to unpack values from arrays and assign them to variables.

```
const [a, b] = [1, 2];
console.log(a); // 1
console.log(b); // 2

  const character = {
    thename: "Gandalf",
    thetitle: "the Grey",
    attributes: {
      race: "Maia",
      age: "Ancient",
    },
    equipment: {
      weapon: "Staff",
      secondaryWeapon: "Sword",
    },
  };
  const {
    thename,
    thetitle,
    attributes: { race, age },
    equipment: { weapon, secondaryWeapon },
  } = character;
  console.log(`${thename}, ${thetitle}, is a ${age} ${race} wielding a ${weapon} and a
${secondaryWeapon}.`);
```

Practice: Write an object named battle with any data you prefer, for example. the location can be Minas Tirith and number of participants 10000. And destructure it, print your result to console.

TASK 1. Run all the code listed, Implement all Practice Exercises. Submit your source code as .txt file and take screenshots of your code and its output.

Part 2: Intro to Asynchronous JS

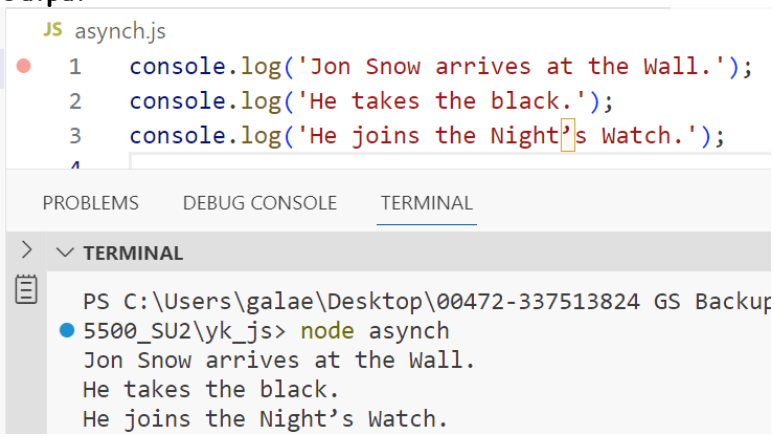
2.1 Fundamentals of Asynchronicity

Synchronous code runs sequentially, blocking the execution of subsequent code until the current operation completes. This can be compared to waiting in a queue at the Wall, where each member of the Night's Watch must wait for their turn.

Crate a file asynch.js and copy-paste snippets below there

```
console.log('Jon Snow arrives at the Wall.');
console.log('He takes the black.');
console.log('He joins the Night's Watch.');
```

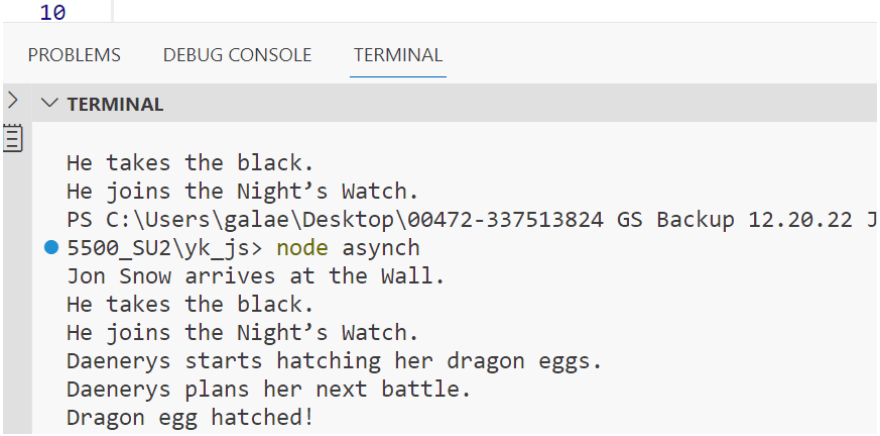
Output:



Asynchronous code allows **multiple operations to run concurrently**, without waiting for previous ones to complete. Imagine Daenerys Targaryen hatching her dragon eggs while strategizing for her next battle.

```
console.log('Daenerys starts hatching her dragon eggs.');
setTimeout(() => {
  console.log('Dragon egg hatched!');
}, 2000);
console.log('Daenerys plans her next battle.');
```

```
5   console.log('Daenerys starts hatching her dragon eggs.');
6   setTimeout(() => {
7     console.log('Dragon egg hatched!');
8   }, 2000);
9   console.log('Daenerys plans her next battle.');
10
```



Practice: change 2 sec to 3, 4 and 0 and see if any difference, record your findings

2.2 Event loop

The **event loop** is the core mechanism behind JavaScript's asynchronous behavior. It continuously checks the call stack and the task queue, executing tasks in a non-blocking manner. Unlike some other languages that use threads to manage multiple tasks, JavaScript uses a **single-threaded event loop** that can handle multiple operations **without blocking** the execution of the code.

Call Stack: This is where JavaScript keeps track of the function calls. When a function is invoked, it is pushed onto the call stack. When the function returns, it pops off the stack.

Web APIs: These are APIs provided by the browser (or Node.js) that can handle operations like setTimeout, DOM events, HTTP requests, etc. When such an operation is invoked, it is handed off to the Web API.

Callback Queue: Once an asynchronous operation is complete (e.g., a timer expires, an HTTP request finishes), its callback function is pushed to the callback queue.

Try this in a separate file

```
console.log('Jon Snow starts his watch.');
setTimeout(() => {
  console.log('Jon Snow spots a White Walker.');
}, 3000);
console.log('Jon Snow patrols the Wall.');
console.log('Jon Snow starts his watch.');
setTimeout(() => {
  console.log('Jon Snow spots a White Walker.');
}, 3000);
console.log('Jon Snow patrols the Wall.');
```

```
JS john.js > setTimeout() callback
1   console.log('Jon Snow starts his watch.');
2   setTimeout(() => {
3     console.log('Jon Snow spots the White Walker.');
4   }, 3000);
5   console.log('Jon Snow patrols the Wall.');
6   console.log('Jon Snow starts his watch.');
7   setTimeout(() => {
8     console.log('Jon Snow spots a White Walker.');
9   }, 3000);
10  console.log('Jon Snow patrols the Wall.');
```

```
PROBLEMS  DEBUG CONSOLE  TERMINAL
TERMINAL
PS C:\Users\galae\Desktop\00472-337513824 GS Backup 12.20.22
5500_SU2\yk_js> node john
Jon Snow starts his watch.
Jon Snow patrols the Wall.
Jon Snow starts his watch.
Jon Snow patrols the Wall.
Jon Snow spots a White Walker.
Jon Snow spots a White Walker.
```

Practice: Imagine there is a grand feast happening at Winterfell. Several key events will take place during the feast, and you'll use JavaScript to simulate these events in a non-blocking manner.

Synchronously log a welcome message for each guest.
Asynchronously serve drinks to each guest after a short delay.
Announce the start of the entertainment program synchronously.
Asynchronously announce the start of the feast after a short delay.
Asynchronously perform three different acts (juggling, singing, and dancing) at different times during the feast.
Synchronously thank the guests for attending the feast.

2.3 Promises

Promises represent the eventual result of an asynchronous operation. They can be in one of three states: **pending**, **fulfilled**, or **rejected**. Promises are a fundamental concept in JavaScript for handling asynchronous operations. They represent a value that might be available now, or in the future, or never. A **Promise** is an **object** representing the **eventual completion or failure** of an asynchronous operation. Promises provide a cleaner, more robust way to handle asynchronous code compared to traditional callback-based approaches. You can create a Promise using the Promise constructor, which takes a function (executor) as an argument. The executor function takes two parameters: resolve and reject.

resolve(value): Called when the asynchronous operation completes successfully.
reject(reason): Called when the asynchronous operation fails. Try in a separate file

```
const feastPromise = new Promise((resolve, reject) => {
  let isFeastReady = true;

  if (isFeastReady) {
    resolve('The feast is ready! Enjoy the meal.');
  } else {
    reject('The feast is not ready yet. Please wait.');
  }
```

```
});

feastPromise
  .then(message => {
    console.log(message); // This will run if the promise is resolved
  })
  .catch(error => {
    console.log(error); // This will run if the promise is rejected
  });
```

Practice: change to false, rerun, then remove catch block, rerun.

Another example to try

```
const getDragons = new Promise((resolve, reject) => {
  setTimeout(() => {
    const dragons = ['Drogon', 'Rhaegal', 'Viserion'];
    resolve(dragons);
  }, 2000);
});

getDragons
  .then((dragons) => {
    console.log('Daenerys has dragons:', dragons);
  })
  .catch((error) => {
    console.error('Failed to get dragons:', error);
  });
```

Practice: change to false, rerun, then remove catch block, rerun.

2.4 Promise Chaining

Promise chaining allows to perform multiple asynchronous **operations in sequence**, with each operation depending on the previous one.

```
const getDragons = new Promise((resolve, reject) => {
    setTimeout(() => {
      const dragons = ['Drogon', 'Rhaegal', 'Viserion'];
      resolve(dragons);
    }, 2000);
  });
  const prepareArmy = new Promise((resolve, reject) => {
    setTimeout(() => {
      const army = ['Unsullied', 'Dothraki', 'Northmen'];
      resolve(army);
    }, 3000);
  });
  const forgeWeapons = new Promise((resolve, reject) => {
    setTimeout(() => {
      const weapons = ['Valyrian Steel Swords', 'Dragonglass Arrows'];
      resolve(weapons);
```

```
    }, 4000);
  });

  console.log('Preparing for the Battle of Winterfell...');

  Promise.all([getDragons, prepareArmy, forgeWeapons])
    .then((results) => {
      const [dragons, army, weapons] = results;
      console.log('Daenerys has dragons:', dragons);
      console.log('Jon Snow has gathered the army:', army);
      console.log('Weapons are ready:', weapons);
      return 'Battle of Winterfell';
    })
    .then((battle) => {
      console.log('Next event:', battle);
    })
    .catch((error) => {
      console.error('Error:', error);
    });
```

Practice: Instructions

Create a promise that simulates gathering magical ingredients with a delay of 2 seconds. Resolve the promise with an array of ingredient names.

Create a promise that simulates preparing the enchanted hall with a delay of 3 seconds. Resolve the promise with a confirmation message.

Create a promise that simulates brewing the potions with a delay of 4 seconds. Resolve the promise with an array of potion names.

Create a promise that simulates sending out magical invitations with a delay of 2 seconds. Resolve the promise with an array of invited wizard names.

2.5 Promise API

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

Promise.all():

```
Waits for all promises to resolve or any to reject.

const promise1 = Promise.resolve('Winter is Coming');
const promise2 = Promise.resolve('The North Remembers');
const promise3 = Promise.resolve('Valar Morghulis');

Promise.all([promise1, promise2, promise3])
  .then((messages) => {
    console.log('Messages:', messages);
  })
  .catch((error) => {
    console.error('Error:', error);
  });
Messages: ['Winter is Coming', 'The North Remembers', 'Valar Morghulis']
```

Promise.race():

Waits for the first promise to resolve or reject.

```
const promise1 = new Promise((resolve) => setTimeout(resolve, 500, 'Arya Stark'));
const promise2 = new Promise((resolve) => setTimeout(resolve, 1000, 'Sansa Stark'));

Promise.race([promise1, promise2])
  .then((winner) => {
    console.log('First to complete:', winner);
  })
  .catch((error) => {
    console.error('Error:', error);
  });
```

2.6. Async/Await

async functions return a Promise, and await pauses execution until the Promise resolves.

```
async function fetchDragons() {
  const dragons = new Promise((resolve) => {
    setTimeout(() => resolve(['Drogon', 'Rhaegal', 'Viserion']), 2000);
  });
  const result = await dragons;
  console.log('Dragons:', result);
}
fetchDragons();
```

2.7. Error Handling

Use try...catch blocks to handle errors in async/await code.

```
async function fetchDragons() {
  try {
    const dragons = new Promise((resolve, reject) => {
      setTimeout(() => reject('Dragons not found'), 2000);
    });

    const result = await dragons;
    console.log('Dragons:', result);
  } catch (error) {
    console.error('Error:', error);
  }
}

fetchDragons();
```

Practice: Preparing for the Quidditch Match. In the magical world of Harry Potter, the Quidditch match is about to begin. As the organizer, you need to ensure everything is ready before the match starts. Create promises to represent each preparation step and use async/await to handle the process.

Steps:
- Create a promise that resolves after 1 second with the message "Quidditch pitch is set up."
- Create a promise that resolves after 2 seconds with the message "Broomsticks are gathered."
- Create a promise that resolves after 3 seconds with the message "Golden Snitch is ready."

Async Function:
- Create an async function prepareQuidditchMatch that uses await to wait for each preparation step.
- Log each step's message to console as it completes.

Error Handling:
- Modify one of the promises to reject with an error message (e.g., "Broomsticks are missing!").
- Use try...catch in the prepareQuidditchMatch function to handle the error gracefully and log an appropriate message.

Practice: Preparing Potions for the Feast
At Hogwarts, the house-elves are busy preparing for the grand feast. Your task is to gather all the necessary potions. Create promises to represent each preparation step and use async/await to handle the process.

Steps:
- Create a promise that resolves after 2 seconds with the message "Felix Felicis is brewed."
- Create a promise that resolves after 3 seconds with the message "Polyjuice Potion is prepared."
- Create a promise that resolves after 1 second with the message "Amortentia is created."
- Create an async function prepareFeastPotions that uses await to wait for each preparation step.
- Log each step's message to console as it completes.
- Modify one of the promises to reject with an error message (e.g., "Polyjuice Potion ingredients are missing!").
- Use try...catch in the prepareFeastPotions function to handle the error gracefully and log an appropriate message.

2.8. Parallel Execution with Promise.all():

async function prepareForBattle() {

```
  const getDragons = new Promise((resolve) => setTimeout(resolve, 2000, ['Drogon', 'Rhaegal', 'Viserion']));
  const gatherArmy = new Promise((resolve) => setTimeout(resolve, 3000, 'Unsullied'));

  const [dragons, army] = await Promise.all([getDragons, gatherArmy]);

  console.log('Dragons:', dragons);
  console.log('Army:', army);
}

prepareForBattle();
```

Sequential Execution with for await...of Loops:

```
async function trainDragons() {
  const dragonNames = ['Drogon', 'Rhaegal', 'Viserion'];

  for (const dragon of dragonNames) {
```

```
    await new Promise((resolve) => setTimeout(resolve, 1000));
    console.log(`${dragon} is trained!`);
  }
}

trainDragons();
```

2.9. Fetch API

Making network requests using the Fetch API.

```
5500_SU2\yk_js> node fetch
Character: Jon Snow
```

```
async function getCharacterData() {
  try {
    const response = await fetch('https://anapioficeandfire.com/api/characters/583');
    const data = await response.json();
    console.log('Character:', data.name);
  } catch (error) {
    console.error('Error:', error);
  }
}

getCharacterData();
```

Practice: update this code to use axios: https://axios-http.com/docs/intro

2.10. Advanced features

Run the code below. Talk to LLMs to get more information

Web Workers - Offload computationally intensive tasks to separate threads.

```
// worker.js
const { parentPort } = require('worker_threads');

parentPort.on('message', (data) => {
  console.log('Message received from main script');
  const result = data[0] * data[1];
  parentPort.postMessage(result);
});
// main.js
const { Worker } = require('worker_threads');

const worker = new Worker('./worker.js');

worker.postMessage([10, 2]);

worker.on('message', (result) => {
  console.log('Result:', result);
});
```

```
worker.on('error', (error) => {
  console.error('Error:', error);
});

worker.on('exit', (code) => {
  if (code !== 0)
    console.error(`Worker stopped with exit code ${code}`);
});
```

```
PS_5500_SU2\yk_jnode main
Message received from main script
Result: 20
```

Event Emitters
Create custom events and manage event-driven logic.

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

emitter.on('knightArrives', (name) => {
  console.log(`${name} has arrived at Winterfell`);
});

emitter.emit('knightArrives', 'Brienne of Tarth');
```

Observables (RxJS)
Handling asynchronous streams of data using Observables.

```
PS_5500_SU2\yk_js> npm install rxjs
```

TASK 2. Run all the code listed, Implement all Practice Exercises. Submit your source code as .txt file and output.

Part 3: Intro to Node.js

3.1 Introduction to Node.js and NPM

Node.js is built on the event-driven, non-blocking I/O model which makes it lightweight and efficient for building scalable network applications. Node.js uses events heavily and it's one of the reasons why Node.js is fast. An event-driven architecture **can process a high number of requests without waiting for any function** to return. Non-blocking I/O means that Node.js doesn't wait for the file reading or writing to finish; instead, **it will continue with the next request**. This is made possible by callbacks and **asynchronous programming**.

Create a file node_module1.js and try the code below:

```
const fs = require('fs');

console.log("Starting the spell...");
```

```
fs.readFile('magic.txt', (err, data) => {
    if (err) {
        console.log("Failed to read the spell");
    } else {
        console.log("Spell content: " + data.toString());
    }
});

console.log("Ready for the next spell!");
```

Practice:

Create a **file** named magic.txt in the same folder with some text in it.

magic.txt

```
1    love potion
```

Rerun the script

npm (Node Package Manager) is the default package manager for Node.js. It allows to **manage project dependencies** and **install packages** from the npm registry.

How to Use npm:

- Create a new folder express, open in VSC, open in integrated terminal

Initialize a Project:

```
PS C:\Users\galae\Desktop\00472-337513824 (
PS_5500_SU2\yk_js> cd express
PS C:\Users\galae\Desktop\00472-337513824 (
PS_5500_SU2\yk_js\express> npm init -y
```

npm init -y

Install express framework:

npm install express

```
PS_5500_SU2\yk_js\express> npm install express
```

Use a Package:

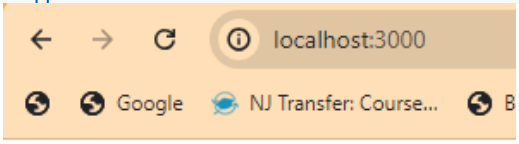
Create an index.js file

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
    res.send('Welcome to Hogwarts!');
});

app.listen(3000, () => {
    console.log('Server running on port 3000');
});
```

```
PS_5500_SU2\yk_js\express> node index
Server running on port 3000
```



Welcome to Hogwarts!

Practice small activity with LLMs:

Ask LLMs like ChatGPT on how to connect to file system from this app and fetch data on the web page from magic.txt

3.2. Express Framework

I Discussed with LLMs how to convert this small app into a fully grown website about Volandemort

Practice small activity From ChatGPT-4o:

Step 1: Create the file structure

```
express/
├── public/
│   ├── css/
│   │   └── styles.css
│   ├── js/
│   │   └── scripts.js
│   └── images/
│
├── views/
│   ├── index.html
│   ├── about.html
│   ├── history.html
│   └── achievements.html
├── index.js
├── package.json
└── package-lock.json
```

Step 2: Create your Views/HTML Files

views/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Voldemort - Home</title>
    <link rel="stylesheet" href="/css/styles.css">
</head>
<body>
    <header>
        <h1>Welcome to the World of Voldemort</h1>
        <nav>
            <ul>
                <li><a href="/">Home</a></li>
                <li><a href="/about">About</a></li>
                <li><a href="/history">History</a></li>
                <li><a href="/achievements">Achievements</a></li>
            </ul>
        </nav>
    </header>
    <main>
        <h2>Introduction</h2>
<p>Discover the life and times of Lord Voldemort, the Dark Lord.</p>
        <img src="https://wallpapercave.com/wp/wp5313132.jpg" alt="Voldemort">
```

```
    </main>
    <script src="/js/scripts.js"></script>
</body>
</html>
```

views/about.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Voldemort - About</title>
    <link rel="stylesheet" href="/css/styles.css">
</head>
<body>
    <header>
        <h1>About Voldemort</h1>
        <nav>
            <ul>
                <li><a href="/">Home</a></li>
                <li><a href="/about">About</a></li>
                <li><a href="/history">History</a></li>
                <li><a href="/achievements">Achievements</a></li>
            </ul>
        </nav>
    </header>
    <main>
        <h2>About Voldemort</h2>
        <p>Learn about the origins and rise of Tom Marvolo Riddle, better known as Lord
Voldemort.</p>
    </main>
    <script src="/js/scripts.js"></script>
</body>
</html>
```

views/history.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Voldemort - History</title>
    <link rel="stylesheet" href="/css/styles.css">
</head>
<body>
    <header>
        <h1>Voldemort's History</h1>
        <nav>
            <ul>
```

```
            <li><a href="/">Home</a></li>
            <li><a href="/about">About</a></li>
            <li><a href="/history">History</a></li>
            <li><a href="/achievements">Achievements</a></li>
        </ul>
    </nav>
</header>
<main>
    <h2>History of Voldemort</h2>
    <p>Explore the dark history of Lord Voldemort, from his early years to his rise to power.</p>
</main>
<script src="/js/scripts.js"></script>
</body>
</html>
```

views/achievements.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Voldemort - Achievements</title>
    <link rel="stylesheet" href="/css/styles.css">
</head>
<body>
    <header>
        <h1>Voldemort's Achievements</h1>
        <nav>
            <ul>
                <li><a href="/">Home</a></li>
                <li><a href="/about">About</a></li>
                <li><a href="/history">History</a></li>
                <li><a href="/achievements">Achievements</a></li>
            </ul>
        </nav>
    </header>
    <main>
        <h2>Achievements</h2>
        <p>Discover the notable achievements and dark deeds of Lord Voldemort.</p>
    </main>
    <script src="/js/scripts.js"></script>
</body>
</html>
```

Step 3: Create Static Files

public/css/styles.css

```
body {
    font-family: Arial, sans-serif;
    background-color: #222;
    color: #eee;
    margin: 0;
```

```
    padding: 0;
}

header {
    background-color: #000;
    color: #fff;
    padding: 10px 0;
    text-align: center;
}

nav ul {
    list-style-type: none;
    padding: 0;
}

nav ul li {
    display: inline;
    margin: 0 10px;
}

nav ul li a {
    color: #fff;
    text-decoration: none;
}

main {
    padding: 20px;
}

img {
    max-width: 100%;
}
```

public/js/scripts.js

```
document.addEventListener('DOMContentLoaded', () => {
    console.log('Page loaded');
});
```

Step 4: Update the Server

index.js

```
const express = require('express');
const path = require('path');
const app = express();

// Serve static files from the "public" directory
app.use(express.static(path.join(__dirname, 'public')));

// Route for home page
app.get('/', (req, res) => {
    res.sendFile(path.join(__dirname, 'views', 'index.html'));
});
```

```
// Route for about page
app.get('/about', (req, res) => {
    res.sendFile(path.join(__dirname, 'views', 'about.html'));
});

// Route for history page
app.get('/history', (req, res) => {
    res.sendFile(path.join(__dirname, 'views', 'history.html'));
});

// Route for achievements page
app.get('/achievements', (req, res) => {
    res.sendFile(path.join(__dirname, 'views', 'achievements.html'));
});

// Catch-all route for 404 - Page Not Found
app.get('*', (req, res) => {
    res.status(404).send('<h1>404 - Page Not Found</h1>');
});

// Start the server
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
});
```

Step 6: Run the Server

To run the server, use the following command:

```
node index.js
```

TASK 3. Run all the code listed, Implement all Practice Exercises. Submit your source code as .txt file and output.